The Chinese Postman Problem — 2026 Revision

Revised edition of the 1997 final-year project by Miquel Monserrat Pons (supervisor: Ramón Companys Pascual)

Nov 1, 2026 · @Miquel

Abstract

This report revises a 1997 final-year project on the Chinese Postman Problem: find the cheapest closed route that covers every street of a network at least once. The method is unchanged in outline: pair the odd-degree junctions by shortest paths at minimum total cost, duplicate those paths, and walk the resulting Eulerian circuit. The 1997 branch-and-bound for the pairing step is replaced by Edmonds' blossom algorithm, which is exact and polynomial; shortest paths use Dijkstra, the circuit uses Hierholzer's linear-time algorithm, the directed version is added through minimum-cost flow, and the mixed version, with one-way and two-way streets together and NP-hard, through branch-and-cut and a new heuristic.

The new Python implementation was tested on classic graphs, on 92 benchmark road networks from the arc-routing literature (including real networks in Cumbria and Flanders), and on synthetic towns of up to 40,000 junctions. Every instance within the memory budget was solved to proven optimality. With a second backend that runs Kolmogorov's Blossom V on a sparse candidate graph and adds pairs by dual pricing, a network with 12,066 odd vertices solves in 102 seconds, against about 800 odd vertices in the same time in pure Python. On benchmarks where every street needs service, the single-postman tour comes within a median 3.4% of the best known multi-vehicle solution, making the CPP a tight bound for fleet routing. For mixed networks, branch-and-cut proves the optimum on every benchmark network within 5.2 seconds and on towns of up to 1,600 junctions, and the heuristic averages 0.27% above the optimum. The report closes with a map of the harder arc-routing variants where the branch-and-bound ideas of 1997 remain useful.

1. Introduction

The Chinese Postman Problem (CPP) asks for the cheapest closed walk that traverses every edge of a graph at least once. The 1997 project solved it by duplicating edges until every vertex has even degree, choosing those duplicates with a custom branch-and-

bound (B&B) search. This revision keeps the same overall method but replaces the B&B with an exact, polynomial-time matching algorithm, and tests the result on a much wider set of instances.

The problem is named after the Chinese mathematician Kwan Mei-Ko (Guan Meigu), who posed it in 1962. Edmonds and Johnson showed in 1973 that the undirected version can be solved in polynomial time by reducing it to a minimum-weight perfect matching.

Typical applications are any service that must cover streets rather than visit points:

- mail and parcel delivery on foot;
- snow ploughing and gritting;
- refuse collection and street sweeping;
- park and street-tree maintenance crews;
- inspection of pipelines, power lines and road markings.

What changed since 1997

Aspect	1997 project	2026 revision
Matching step	Binary B&B over edges, lower bound from row minima	Edmonds' blossom algorithm (exact, polynomial)
Shortest paths	Fulkerson iteration on the full V × V matrix	Dijkstra from each odd vertex only
Eulerian circuit	Cycle splicing with linear scans of adjacency queues	Hierholzer's algorithm, O(m)
Directed graphs	Proposed as future work	Solved by minimum-cost flow
Language and platform	Turbo C, MS-DOS, 640 KB memory model	Python 3 with NetworkX, any platform
Practical limit	About 20 odd vertices to proven optimality	About 12,000 odd vertices (40,000 junctions) in under two minutes with Blossom V; about 800 in pure Python
Mixed streets (one-way and two-way)	Not covered	Exact branch-and-cut (HiGHS) and a heuristic with local search

Contributions of this revision

1. A clean, tested reimplementation of the full CPP pipeline.

2. A computational study on six families of instances, none taken from the original Vilanova i la Geltrú case studies.
3. A comparison of the 1997 branch-and-bound with the exact method, and a tightened lower bound for anyone who extends the B&B to harder variants.
4. Extensions to directed and mixed networks and a map of the wider arc-routing field for future work.

2. Graph-theoretic background

Only four ideas are needed: graphs with edge costs, vertex degree, Euler's theorem and matchings. The 1997 report spent a chapter on general nomenclature; this revision keeps just what the method uses.

Graph. $G = (V, E)$ has a vertex set V and an edge set E, with $n = |V|$ and $m = |E|$. In an undirected graph each edge joins two vertices without orientation; in a directed graph each arc goes from a tail to a head. A multigraph allows several parallel edges between the same pair. Each edge e carries a cost $c(e) > 0$, such as length, time or fuel.

Degree. The degree of a vertex is the number of edge ends that meet it. Because every edge has two ends, the sum of all degrees is $2m$, so every graph has an even number of odd-degree vertices. This handshake lemma is why the odd vertices can always be paired up.

Walks and circuits. A walk is a sequence of edges in which consecutive edges share an end. A closed walk returns to its start. An Eulerian circuit is a closed walk that uses every edge exactly once.

Euler's theorem. A connected multigraph has an Eulerian circuit if and only if every vertex has even degree. For a strongly connected directed multigraph, the condition is that every vertex has in-degree equal to out-degree.

Matchings. A matching is a set of edges no two of which share a vertex. It is perfect when it covers every vertex. Given edge costs, a minimum-weight perfect matching is a perfect matching of least total cost. On a complete graph with an even number of vertices a perfect matching always exists.

Shortest paths. $d(u, v)$ is the cost of a cheapest path from u to v. With non-negative costs it is computed by Dijkstra's algorithm in $O(m + n \log n)$ time per source with a binary heap.

3. The Chinese Postman Problem

The optimal postman tour costs the sum of all edges plus the cheapest way to pair up the odd vertices by shortest paths. Everything else in this report is about computing that pairing and then walking the resulting graph.

3.1 Formulation

Given a connected undirected multigraph G = (V, E) with costs c(e) > 0, find a closed walk that traverses every edge at least once and has minimum total cost. Equivalently, choose how many extra copies x(e) ⩾ 0 of each edge to add so that the augmented multigraph is Eulerian, at minimum added cost:

```
\min \sum_{e \in E} c(e)\,x(e) \quad \text{s.t.} \quad \deg_G(v) + \sum_{e \ni v} x(e) \equiv 0 \pmod 2 \;\; \forall v \in V, \qquad x(e) \in \mathbb{Z}_{\ge 0}
```

An optimal solution never uses more than one extra copy of an edge, so x(e) ∈ {0, 1}. The set of duplicated edges is a minimum-cost T-join, where T is the set of odd vertices: a set of edges in which exactly the vertices of T have odd degree.

3.2 Edmonds' method

1. Let T be the set of odd-degree vertices. If T is empty, G is already Eulerian; go to step 5.
2. Compute shortest-path distances d(u, v) for every pair u, v in T.
3. Build the complete graph K_T on T with edge costs d(u, v), and find a minimum-weight perfect matching M* on it.
4. For each pair (u, v) in M*, duplicate every edge on a shortest u–v path in G.
5. The augmented multigraph has all degrees even; find an Eulerian circuit in it. That circuit is an optimal postman tour.

The total cost is c(E) + c(M*). The method is exact because the shortest paths of an optimal matching never need to share an edge: if two paths did, removing both copies of the shared edge would keep all degrees' parity and lower the cost.

3.3 Worked example from 1997

The 1997 report used a small park-maintenance network with 8 vertices and 12 edges. Its edge costs, taken from the street plan, are:

Edge	1–8	1–7	8–7	1–4	7–4	7–6	4–5	6–5	6–3	3–5	5–2	3–2
Cost	2	1	5	6	4	7	2	1	4	7	6	9

As in the original report, costs stand for travel time or fuel rather than plain distance.

The odd vertices are T = {1, 3, 4, 6}. The three possible perfect matchings on K_T cost:

Matching	Cost
(1,3) + (4,6)	12 + 3 = 15
(1,4) + (3,6)	5 + 4 = 9
(1,6) + (3,4)	8 + 7 = 15

The optimum is M* = {(1,4), (3,6)}. It duplicates the path 1–7–4 and the edge 3–6. The original network costs 54, so the optimal tour costs 54 + 9 = 63 and has 15 edges. The 2026 code reproduces this exactly and returns the tour 1–8–7–1–7–4–7–6–3–2–5–3–6–5–4–1.

4. Minimum-weight perfect matching

The matching step is solved exactly, in polynomial time, by Edmonds' blossom algorithm. This replaces the 1997 branch-and-bound entirely: there is no longer any instance size at which the method has to fall back on a time limit or a heuristic.

4.1 Linear programming formulation

Let K = (T, F) be the complete graph on the odd vertices, with cost d(e) on each edge, and let x(e) = 1 when edge e is in the matching. For any vertex set S, δ(S) is the set of edges with exactly one end in S. Edmonds (1965) showed that the following linear program has an integral optimum, so it describes perfect matchings exactly:

```
\min \sum_{e \in F} d(e)\,x(e) \quad \text{s.t.} \quad x(\delta(v)) = 1 \;\;
\forall v \in T, \qquad x(\delta(S)) \ge 1 \;\; \forall S \subseteq T,\ |S|
\ge 3 \text{ odd}, \qquad x \ge 0
```

Without the odd-set constraints the program can return half-integral solutions on odd cycles: three vertices joined in a triangle, each edge at 0.5. The odd-set constraints rule these out. There are exponentially many of them, but the algorithm never lists them; it only touches the few that matter.

4.2 How the blossom algorithm works

The algorithm is a primal-dual method. It keeps a matching, which starts empty, and a feasible dual solution: a price y(v) on each vertex and a price z(B) on each odd set B in use. An edge is tight when the prices of its ends, plus those of the odd sets it crosses, equal its cost. The algorithm repeats three moves until every vertex is matched:

1. **Grow.** From an unmatched vertex, build an alternating tree using tight edges only. If it reaches another unmatched vertex, flip the path between them: the matching grows by one edge.
2. **Shrink.** If the tree closes an odd cycle (a blossom), contract the cycle into a single pseudo-vertex and keep growing.
3. **Adjust prices.** If the tree is stuck, change the dual prices by the largest step that keeps them feasible. This makes at least one new edge tight, or lets a pseudo-vertex be expanded again.

When every vertex is matched, the matching and the prices satisfy complementary slackness, which proves optimality. Chapter 3 of the 1997 report already presented this algorithm. The 2026 version uses it directly, through a mature library implementation, so the matching no longer needs a purpose-built search.

4.3 Complexity and implementations

Implementation	Language	Worst-case time	Notes
Edmonds (1965)	—	$O(n^4)$	Original algorithm
Gabow (1976), Lawler (1976)	—	$O(n^3)$	Standard textbook bound
Galil, Micali and Gabow (1986)	—	$O(nm \log n)$	Better on sparse graphs
Blossom V, Kolmogorov (2009)	C++	$O(n^3)$ worst case, much faster in practice	Fast backend in this revision; non-commercial licence
NetworkX `min_weight_matching`	Python	$O(n^3)$	Default backend in this revision

Here n is the number of odd vertices and m the number of edges of K, so $m = n(n - 1)/2$. NetworkX was chosen for readability and portability: the whole solver is under 150 lines of Python. Its pure-Python matching becomes the bottleneck above a few hundred odd vertices (section 7.4), so large networks use a second backend built on Blossom V, described next.

4.4 Large networks: a sparse candidate graph with dual pricing

The complete graph K has about $n^2/2$ edges, yet an optimal matching almost always pairs each odd vertex with one of its nearest odd neighbours. The fast backend exploits this without giving up the optimality guarantee:

1. Compute shortest-path distances from every odd vertex with SciPy's compiled Dijkstra.
2. Build a sparse candidate graph: each odd vertex's 10 nearest odd vertices, plus the pairs of a greedy matching so that a perfect matching is sure to exist.
3. Solve it with Blossom V, Kolmogorov's C++ implementation, and read the optimal dual prices y.
4. Pricing: scan all pairs (u, v) not in the graph and add those with negative reduced cost, d(u, v) < y(u) + y(v), counting blossom prices. Re-solve and repeat.
5. When no pair has negative reduced cost, the duals are feasible for the whole complete graph, so the matching is optimal for it too.

This is column generation applied to matching, the same strategy Blossom V uses for its geometric instances. On road networks the first candidate graph is usually already enough: one or two pricing rounds suffice. Related ideas reach even further: the sparse blossom algorithm of Higgott and Gidney (2023), developed for quantum error correction, matches millions of vertices without ever building distances between all pairs.

Blossom V may be used for evaluation and non-commercial research only and may not be redistributed. The package therefore ships only a small C wrapper and a script that downloads and compiles it; NetworkX stays the default backend, and commercial use would need a licence from UCL Business or a free alternative such as LEMON (Boost licence).

5. Eulerian circuits

Once every degree is even, Hierholzer's algorithm (1873) writes out the tour in time proportional to the number of edges. It is the algorithm the 1997 project already sketched in its section 5.2.2, now implemented so that each edge is looked at once.

The idea follows the constructive proof of Euler's theorem. Walk from the start vertex along unused edges until you are stuck; with all degrees even, you can only get stuck back at the start, so you have a closed sub-tour. Then return along that sub-tour to the first vertex that still has unused edges, walk a new sub-tour from there and splice it in. Repeat until no unused edges remain.

The iterative version uses a stack:

1. Push the start vertex.
2. Look at the vertex v on top of the stack. Advance v's pointer past any edges already used.
3. If v has an unused edge (v, w), mark it used and push w.
4. Otherwise pop v and append it to the output.

5. When the stack is empty, the output reversed is the Eulerian circuit.

Each vertex keeps a pointer into its adjacency list that only moves forward, and each edge is marked used once. The total work is therefore O(n + m). This per-vertex pointer is the refinement that makes the 2026 implementation strictly linear in the size of the graph.

Fleury's rule, discussed in the 1997 report, also works but must test at each step whether an edge is a bridge of the remaining graph. Even with modern dynamic connectivity structures this is slower and more complicated than Hierholzer, so it is no longer used in practice.

6. Implementation

The solver is a small Python 3 package, `cartero`, built on NetworkX and covered by 157 automated tests (102 when the optional Blossom V library is not built). It replaces the six Turbo C modules of 1997 (about 1,000 lines plus custom tree, list and matrix structures) with six modules of plain Python and an optional compiled matching library.

6.1 Modules

Module	Replaces (1997)	Responsibility
`io.py`	`GRAFO.C`, `leer_archivo`	Reads and writes the original `.GRA` format; reads the CARP `.dat` benchmark format
`cpp.py`	`MAIN.C`, `FULKER.C`, `ARBOL.C`, `SIMETR.C`	Undirected CPP (Edmonds' method) and directed CPP (minimum-cost flow)
`fast.py`	— (new)	Large networks: SciPy Dijkstra and Blossom V with dual pricing, called through a small C wrapper in native/
`mixed.py`	— (new)	Mixed CPP: bounds, exact branch-and-cut (HiGHS via SciPy), heuristic with local search, mixed instance generators
`euler.py`	`EULERIAN.C`	Hierholzer's algorithm on multigraphs, directed or not
`instances.py`	Annex 2 data	Generators for the test families of chapter 7

Graphs are NetworkX multigraphs, so parallel streets between the same two junctions are kept, as the 1997 adjacency lists allowed. When computing shortest paths only the cheapest of any parallel edges matters, and the solver works on that simple graph internally.

6.2 Pipeline

1. Check that the graph is connected and collect the set T of odd-degree vertices.
2. Run Dijkstra from each vertex of T, keeping distances and paths to the other vertices of T: O(|T| (m + n log n)).
3. Build the complete graph on T and call `min_weight_matching`.
4. Add one copy of every edge on the shortest path of each matched pair.
5. Run Hierholzer on the augmented multigraph and return the tour.

The result object holds the original, added and total costs, the duplicated edges, the matched pairs, the tour as a sequence of vertices, and the time spent in each phase.

6.3 Directed graphs

For a directed graph the imbalance of each vertex, out-degree minus in-degree, plays the role of odd degree. Vertices with more arcs in than out must send extra walks to vertices with more arcs out than in. Choosing those walks at least cost is a transportation problem, solved with NetworkX's `min_cost_flow` on the street network itself, with unbounded arc capacities. The flow on each arc is the number of extra copies to add. The graph must be strongly connected.

6.4 Testing

Test group	Cases	What it checks
1997 example	1	Added cost 9, total 63, matching {(1,4), (3,6)}
Classic graphs	3	Königsberg, Petersen and dodecahedron give the known optimum
Brute force	30	Random graphs with up to 8 odd vertices: same cost as enumerating every pairing
Benchmark bound	23	On gdb, where every edge is required, the CPP never exceeds the published CARP lower bound
Structure	4	Parallel edges, generated cities and directed grids give valid closed tours

Test group	Cases	What it checks
Blossom V backend	55	Same optimum as NetworkX on 51 benchmark files, and on towns where a one-neighbour start forces several pricing rounds
Mixed CPP	41	Exact equals brute-force enumeration on 25 random networks; bound ⩽ optimum ⩽ heuristic; all-two-way and all-one-way cases equal the pure CPPs; the combined strategy proves optimality, including through the cutoff

Every test also verifies the tour itself: it is closed, it covers every original edge, and its length equals the original edges plus the duplicates.

6.5 Usage

```
from cartero import read_gra, solve_undirected

sol = solve_undirected(read_gra("exemples/pfc_1997.gra"), start=1)
print(sol.summary())
```

7. Computational experiments

Every instance tested, 92 distinct real and benchmark road networks and synthetic cities with up to 40,000 junctions, was solved to proven optimality, apart from one 40,000-junction town stopped at the memory limit (section 7.4). None comes from the 1997 Vilanova i la Geltrú case studies. All runs used one core of a 2026 cloud Linux container, Python 3 and NetworkX 3.7.

7.1 Instance families

Family	Source	Networks	Junctions	Streets	Odd vertices
Classic graphs	Königsberg, Petersen, dodecahedron, 1997 example	4	4–20	7–30	4–20
gdb	Golden, DeArmon and Baker (1983), random networks	23	7–27	11–55	0–16
val	Benavent et al. (1992), generated networks	10	24–50	34–97	12–28
kshs	Kiuchi et al. (1995)	6	6–10	15	4–6

Family	Source	Networks	Junctions	Streets	Odd vertices
egl	Li and Eglese (1996), Brandão and Eglese (2008): winter-gritting roads in Cumbria, UK	3	77–255	98–375	50–190
Beullens C, E (D, F share their networks)	Beullens et al. (2003): road network of Flanders, Belgium	50	26–97	35–142	14–54
Grid city	Square grid, short blocks and long avenues, 10% of segments closed	13	100–40,000	161–71,752	42–12,066
Delaunay town	Random junctions, Delaunay triangulation thinned to 75%	11	100–20,000	214–44,974	48–10,006
One-way grid	Directed grid, alternating one-way streets, two-way outer ring	5	48–3,600	113–8,142 arcs	18–204 unbalanced

Benchmark files that share a road network and differ only in demands (for example val1A, val1B and val1C) were solved once, leaving 92 distinct networks out of 254 files. Families A and B in the same repository, of unknown origin, were left out.

7.2 Classic graphs

Graph	Odd vertices	Streets	Added	Total	Tour length
1997 example	4	12	9	63	15 edges
Königsberg bridges	4	7	2	9	9 crossings
Petersen graph	10	15	5	20	20 edges
Dodecahedron	20	30	10	40	40 edges

Euler's citizens of Königsberg could not cross each bridge exactly once, but a postman can cover all seven with nine crossings by repeating two. In the Petersen graph and the dodecahedron every vertex is odd, and both contain a perfect matching of unit edges, so the tour repeats exactly half the edge count.

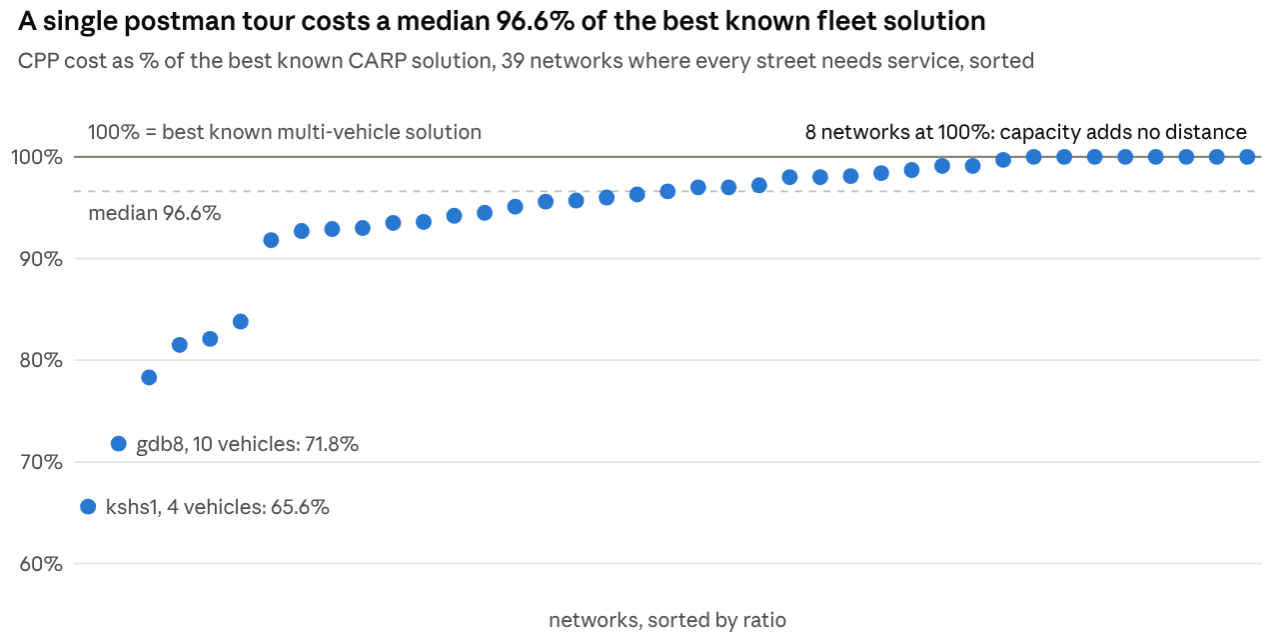
7.3 Benchmark road networks

Family	Networks	Added distance, % of network (mean)	Range	Slowest solve
gdb	23	8.9%	0–22.2%	0.003 s
val	10	14.9%	10.9–18.5%	0.007 s

Family	Networks	Added distance, % of network (mean)	Range	Slowest solve
kshs	6	14.9%	10.5–20.4%	0.001 s
Beullens C	25	20.1%	11.4–24.3%	0.06 s
Beullens E	25	19.4%	12.3–26.9%	0.06 s
egl-e	1	37.4%	—	0.04 s
egl-s	1	24.5%	—	0.21 s
egl-g	1	24.4%	—	2.46 s

On real road networks a postman must repeat about a fifth of the distance; on the sparse Cumbrian network of egl-e, more than a third. The largest benchmark, egl-g, with 255 junctions and 190 odd vertices, takes 2.5 seconds, with optimality proven.

In 39 of these networks every street needs service, so any multi-vehicle CARP solution is also a postman tour, and the CPP optimum is a lower bound for it. The solver respects that bound on all 39. More usefully, the bound is tight:



results/benchmarks.csv · 39 all-required networks from gdb, val and kshs; best known CARP solutions as published in CARPData.jl

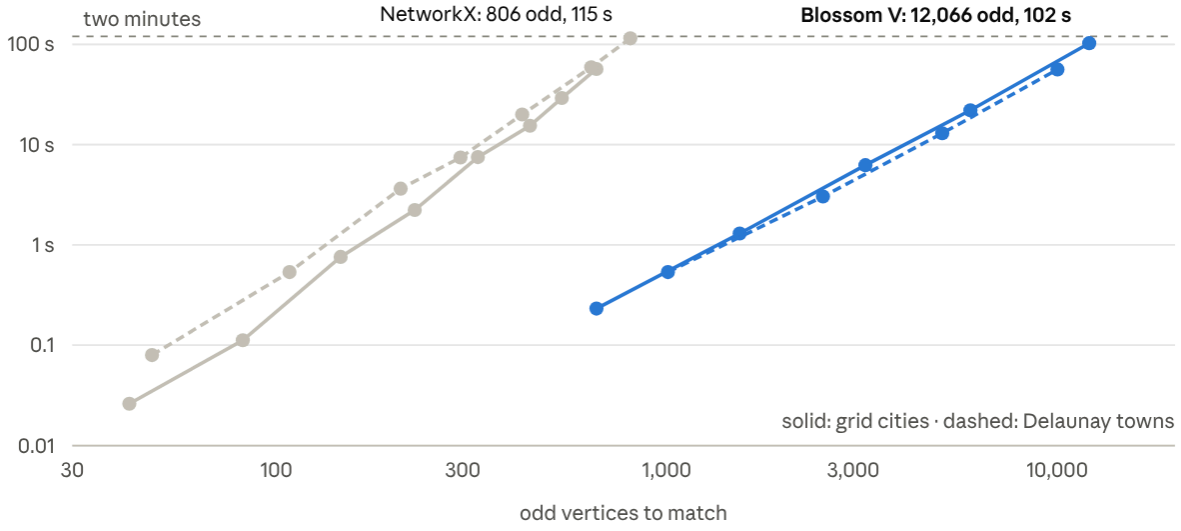
For most networks, splitting the work among several vehicles of limited capacity costs only a few percent more than one unlimited postman. The gaps appear where many vehicles share a small network (gdb8 and gdb9 with 10 vehicles), because every vehicle must drive out from and back to the depot.

7.4 Scaling

With the Blossom V backend, a network with 40,000 junctions and 12,066 odd vertices solves to proven optimality in 102 seconds, against about 800 odd vertices in the same time with NetworkX. Grid cities and Delaunay towns were generated with up to 40,000 and 20,000 junctions respectively; both backends were run, NetworkX until a case passed 90 seconds.

With Blossom V, two minutes now covers 12,000 odd vertices instead of 800

Total solve time against odd vertices, both axes log scale; grid cities and Delaunay towns, one core



results/scaling.csv (NetworkX, 15 networks) and results/scaling_blossom5.csv (Blossom V, 9 networks) · 100–40,000 junctions

Network	Junctions	Odd vertices	Shortest paths	Matching	Tour	Total	Pricing rounds	Peak memory
Grid city	2,025	660	0.1 s	0.01 s	0.1 s	0.2 s	1	0.1 GB
Grid city	10,000	3,224	3.9 s	0.2 s	2.1 s	6.2 s	2	0.4 GB
Delaunay town	10,000	5,072	8.0 s	0.8 s	4.1 s	13 s	3	not measured separately
Grid city	19,600	5,988	14.4 s	0.8 s	6.8 s	22 s	2	1.1 GB
Delaunay town	20,000	10,006	35.4 s	2.7 s	18.0 s	56 s	2	not measured separately
Grid city	40,000	12,066	64.7 s	4.7 s	33.1 s	102 s	3	3.4 GB

On the 660-odd-vertex grid the fast backend is 245 times faster than NetworkX, and the matching itself takes 0.01 s instead of 52 s. Starting from 10 nearest neighbours, one to three pricing rounds were always enough. The matching is no longer the bottleneck: shortest paths take about 60% of the time and building the tour about 30%, both still written in Python around compiled kernels and both easy to speed up further. Memory is dominated by the table of distances between odd vertices, which grows with their square; at 7 GB of RAM this sets the practical ceiling near 20,000 odd vertices, or roughly 50,000–60,000 junctions. A Delaunay town of 40,000 junctions (about 20,000 odd vertices) did not finish within 15 minutes and was stopped. Solving bigger networks would mean never storing that table, as the sparse blossom method does.

For the NetworkX backend, time grows roughly with the cube of the number of odd vertices, as expected from the $O(n^3)$ matching, which takes over 85% of the time above 200 odd vertices.

The repeated distance settles at 12–15% of the network on both families, lower than on the benchmark networks because grids and triangulations are denser.

7.5 Directed networks

Grid	Junctions	Arcs	Unbalanced junctions	Added distance	Solve time
6 × 8	48	113	18	11.7%	0.003 s
10 × 12	120	280	32	10.4%	0.006 s
20 × 20	400	912	64	8.4%	0.02 s
40 × 40	1,600	3,627	134	6.9%	0.14 s
60 × 60	3,600	8,142	204	6.6%	0.39 s

The directed CPP is much faster than the undirected one at the same size because minimum-cost flow works on the sparse street network itself rather than on a complete graph of odd vertices. In a city of alternating one-way streets, a postman repeats only 7–12% of the distance.

8. From branch-and-bound to blossom: improvements in this version

The 2026 version keeps the structure of the 1997 method and improves each of its stages. The largest gain comes from solving the matching exactly in polynomial time; a tightened lower bound also makes the branch-and-bound itself much faster, which matters for the harder variants of chapter 9 where B&B remains the method of choice.

8.1 Summary of improvements

Stage	1997 version	2026 version	Effect
Matching	Binary branch-and-bound with a time limit for large cases	Edmonds' blossom algorithm	Proven optimum at every size, polynomial time
B&B lower bound	Row minima divided by half the number of odd vertices	Row minima divided by 2	Far fewer nodes; useful for B&B on harder variants
Shortest paths	Fulkerson iteration on the full V × V matrix	Dijkstra from odd vertices only	Work proportional to the odd vertices, not to all pairs
Eulerian circuit	Cycle splicing over adjacency queues	Hierholzer with one pointer per vertex	Linear time
Directed streets	Future work	Minimum-cost flow	Polynomial, solved
Platform	Turbo C, 16-bit integers, 640 KB memory model	Python 3, unbounded integers, automatic memory	No size limits on costs or data
Verification	Manual checks	157 automated tests, every tour validated	Results checked on each run

8.2 The 1997 branch-and-bound

Each node of the search tree fixed some pairs of odd vertices as taken and others as rejected. Branching took the cheapest pair still available and created two children: take it or reject it. The lower bound of a node was $c_1 + c_2$, where c_2 is the cost of the pairs already taken and c_1 estimates the cost of completing the matching from the cheapest available pair at each remaining vertex. Nodes were explored lowest bound first while the tree was small, and deepest first afterwards, which gives good solutions early and makes the method a natural time-limited heuristic.

8.3 A tightened lower bound

Every matched pair is reached from both of its ends, so the cost of any perfect matching M is half the sum, over all odd vertices, of the pair each one belongs to. That gives the bound

```
c(M) = \tfrac{1}{2} \sum_{v \in T} d\big(v, M(v)\big) \;\ge\; \tfrac{1}{2}
\sum_{v \in T} \min_{u \ne v} d(u, v)
```

The 1997 version divided the same sum by half the number of odd vertices, imp/2. The two coincide when there are four odd vertices, as in the worked example; with more odd vertices the divisor 2 gives a higher, tighter bound and prunes much more of the tree.

Both bounds were run on the same synthetic street grids, with a limit of 15 seconds or 200,000 nodes per case:

Odd vertices	Nodes, 1997 bound	Nodes, tightened bound
8	82	12
12	8,210	68
16	218,940	65
20	limit reached	637
24	limit reached	3,925
30	limit reached	7,217

With the tightened bound, branching on the most constrained vertex and a starting solution from a greedy pairing improved by pair swaps, the B&B proves optimality up to about 50 odd vertices within 10 seconds. Blossom solves the same cases in under 0.1 s, which is why the 2026 version uses it for the CPP and keeps the B&B for problems with no polynomial algorithm.

8.4 Why the B&B still matters

The 1997 conclusion, that a time-limited B&B yields very good solutions quickly on large instances, still holds and carries over to NP-hard arc routing problems such as the mixed CPP and the rural postman problem. For those, the improvements above (a tight bound, a good first solution and a branching rule that fixes the hardest decision first) are exactly the ingredients modern exact methods rely on.

9. The mixed Chinese Postman Problem

Real towns mix one-way and two-way streets. This mixed CPP is NP-hard (Papadimitriou, 1976), so no polynomial algorithm is known; the 2026 version solves it exactly by branch-and-cut and approximately by a fast heuristic whose distance from the optimum is measured.

9.1 Why it is harder

The two pure cases are easy for different reasons. An undirected network needs only even degrees, fixed by a matching (chapter 3); a directed one needs only as many arcs in as out at each junction, fixed by a minimum-cost flow (chapter 6). In a mixed network the direction in which each two-way street is driven and the choice of streets to repeat must be decided together, because each decision changes the other's constraints.

A mixed network has a postman tour only if it is strongly connected when two-way streets count in both directions. The generators below make sure of it.

9.2 Model and exact method

Let x(u→v) be the number of times the tour drives a street from u to v:

```
\min \sum c\,x \quad \text{s.t.} \quad \sum_{\text{out of } v} x =
\sum_{\text{into } v} x \;\; \forall v, \qquad x(u \to v) \ge 1 \;\text{for a
one-way street}, \qquad x(u \to v) + x(v \to u) \ge 1 \;\text{for a two-way
street}, \qquad x \in \mathbb{Z}_{\ge 0}
```

Any integer solution is a balanced, connected multigraph, so it contains an Eulerian circuit: the tour. The integer program is solved with HiGHS, an open-source solver bundled with SciPy, which runs branch-and-bound with cutting planes: the 1997 idea of separating and bounding, with the bounds supplied by linear programming.

9.3 Bounds and heuristic

Two lower bounds are cheap to compute. The linear relaxation of the model ignores parity and lets a two-way street be driven half each way; the undirected CPP of the network ignores directions. Each misses what the other captures, so the larger of the two is used.

The heuristic is polynomial and runs in four steps:

1. Build four starting orientations of the two-way streets: two roundings of the linear relaxation, and the direction in which an Eulerian circuit of the undirected CPP solution drives each street, read forwards and backwards.
2. For each orientation, add the cheapest extra traversals that balance every junction. With directions fixed this is a minimum-cost flow, solved exactly.
3. Improve the two best by local search. The flow's dual prices m estimate what reversing a street a→b would change, 2·(m_b − m_a); the most promising reversals are tried, each verified by an exact re-solve, and kept only if they lower the cost.
4. Return the best tour found, with the best lower bound as a certificate of how far it can be from the optimum.

The orientation-then-balance idea goes back to Frederickson (1979). The local search is what makes it accurate: on small towns it cut the worst gap to the optimum from 13.6% to 3%.

9.4 Results

On real-size networks the mixed CPP is solved exactly in seconds; on large towns the heuristic comes within about 1% of the best solution found by branch-and-cut, in a fraction of its time.

Benchmark road networks. In each of the 92 distinct networks of chapter 7, 40% of the streets were made one-way in a random direction, keeping the network strongly connected.

Result	Value
Networks solved to proven optimality	92 of 92
Slowest exact solve	5.2 s (egl-g, 255 junctions, 375 streets)
Heuristic finds the optimum	78 of 92
Heuristic gap to optimum	mean 0.12%, maximum 2.8%
Slowest heuristic run	3.3 s

Mixed towns. Grid towns in which each whole street is one-way with probability 0.6, as in a typical town centre, from 64 to 6,400 junctions; one to three towns per size, a 5-minute limit for the exact method.

Junctions	Streets (two-way + one-way)	Exact: time	Exact: proven optimal	Heuristic: time	Heuristic: gap to exact
64	112	1.2–1.4 s	yes	0.2–0.4 s	0–2.6%
144	264	3.3–4.4 s	yes	0.2–0.3 s	0.2–0.9%
256	480	2.0–2.6 s	yes	0.4–0.7 s	0.7–2.8%
400	760	1.3–4.2 s	yes	0.5–0.8 s	0.5–1.8%
900	1,740	8 s	yes	3.9 s	0.8%
1,600	3,120	48 s	yes	18 s	4.8%
3,600	7,080	5 min limit	no; within 0.65% of its bound	22 s	1.0% above the exact incumbent
6,400	12,640	5 min limit	no; within 0.69% of its bound	68 s	1.0% above the exact incumbent

Three conclusions follow.

1. **Exact is practical up to a district.** Branch-and-cut proves the optimum within a minute for towns up to 1,600 junctions and every benchmark network. Above that it still returns a solution within 0.7% of optimal in five minutes, with a certificate.
2. **The heuristic is accurate and scales.** Across the 106 instances with a proven optimum it averages 0.27% above it. Its worst cases (2.8% and 4.8%) are towns where single-street reversals get stuck; moves that reverse whole cycles of two-way streets would be the next improvement.
3. **Bounds matter.** The cheap combined bound averages 94.8% of the optimum on the benchmarks, as low as 83%, so on its own it certifies little. The bound computed by branch-and-cut is far tighter: it is what turns a good solution into a certified one.

9.5 Combined strategy

In practice the two methods are used together, through a single function, `solve_mixed`, that returns the best tour found and a proven lower bound:

1. Compute the cheap bounds and run the heuristic, which gives a good tour in seconds.
2. If the heuristic already matches the bound, or is within a chosen tolerance of it, stop.
3. Otherwise run branch-and-cut for the remaining time, asking it only for tours cheaper than the heuristic's. If none exists the model is infeasible, which proves the heuristic tour optimal; if one exists, it replaces the heuristic tour.

The objective cutoff lets branch-and-cut discard every branch that cannot beat the heuristic, so the heuristic's quality speeds up the exact search. On the 64-junction town where the heuristic was already optimal, this proves it in under a second. On the 1,600-junction town where the heuristic alone was 4.8% above the optimum, the combination returns the optimum in 66 seconds. With a tolerance of 0.5%, small towns stop in under two seconds with a tour certified within that margin.

This is the approach the 1997 report proposed in its last chapter, a time-limited exact search used as a heuristic with a guarantee, now with a branch-and-cut solver behind it.

10. Extensions

The undirected, directed and mixed CPP are now solved; the problems that real fleets face lie one or two steps further, and most of them are NP-hard. The 1997 conclusions anticipated two of them, directed streets and costs that differ between the first and later passes; both turn out to be central to modern arc routing.

Problem	What it adds	Complexity	Typical method
Undirected CPP	—	Polynomial	Matching (this report)
Directed CPP	One-way streets only	Polynomial	Minimum-cost flow (this report)
Mixed CPP	Both one-way and two-way streets	NP-hard	Branch-and-cut and heuristic (chapter 9)
Windy postman	Cost depends on direction (uphill, wind)	NP-hard	Branch-and-cut
Rural postman (RPP)	Only some streets need service	NP-hard	Branch-and-cut; heuristics
Hierarchical CPP	Some streets must be served first	Varies	Exact for special cases
k-CPP (min-max)	k postmen, balance the longest tour	NP-hard	Heuristics, B&B
Capacitated arc routing (CARP)	Several vehicles with limited capacity, a depot	NP-hard	Metaheuristics, branch-cut-and-price
Turn penalties	U-turns and left turns cost extra	Depends on base problem	Transform to a node-routing problem

First and later passes. The 1997 idea of a street costing more the first time it is traversed (snow still on it, bins still full) is exactly the split between service cost and deadheading cost used in the rural postman problem and the CARP. A street is served once at its service cost; any further traversal costs only its travel time.

From one postman to a fleet. Chapter 7 shows that on the classical CARP benchmarks a single CPP tour is usually within a few percent of the best known multi-vehicle solution. The CPP is thus a strong lower bound and a useful first step: split the tour into pieces that respect vehicle capacity (the route-first, cluster-second approach), then improve them locally.

Real street data. OpenStreetMap now provides free, worldwide street networks. The OSMnx library downloads a town's drivable streets as a NetworkX multigraph with lengths and one-way flags, which `cartero` can take after a short conversion (lengths to weights, two-way streets to undirected edges). A district such as the ones studied in 1997 can be modelled in minutes instead of digitised by hand.

Suggested next steps for this project:

1. Cycle-reversal moves for the mixed CPP heuristic, to remove its few remaining gaps of 3–5%.
2. A rural postman variant on the Eglese instances, whose road networks already mark which streets need service.
3. Route-first, cluster-second CARP heuristics, measured against the published bounds in the same benchmark files.

11. Conclusions

The Chinese Postman Problem can now be solved exactly, in polynomial time, on any network a single service crew would plausibly cover. The 2026 version keeps the method of 1997 (pair the odd vertices by shortest paths, duplicate those paths, walk the Eulerian circuit) and improves every stage of it.

1. **Exact matching.** Edmonds' blossom algorithm, through NetworkX, gives the proven optimum on every instance tested: 4 classic graphs, 92 benchmark road networks from the UK, Belgium and the arc-routing literature, and 15 synthetic towns.
2. **Practical size.** With the Blossom V backend, a 40,000-junction network with 12,066 odd vertices solves in 102 seconds; in pure Python the same time covers about 800 odd vertices. Every benchmark road network solves in under 2.5 seconds. Beyond about 20,000 odd vertices, memory for the distance table becomes the limit.
3. **Directed streets.** The directed CPP, proposed as future work in 1997, is solved by minimum-cost flow in under half a second on a 3,600-junction one-way grid.
4. **A strong bound for fleets.** On 39 benchmark networks where every street needs service, the single-postman tour costs a median 96.6% of the best known multi-vehicle solution. The CPP is therefore both a quick estimate and a tight lower bound for capacitated arc routing.
5. **Mixed streets.** The mixed CPP, NP-hard, is solved exactly by branch-and-cut on every benchmark network (at most 5.2 s) and on towns of up to 1,600 junctions; beyond that a polynomial heuristic with dual-guided local search stays about 1% from the best known solution. Separation and bounding, the core of the 1997 work, is exactly what makes this possible.

The two observations of 1997 about practice still hold. Routine services such as refuse collection or mail delivery are bounded by a working shift, so the networks involved stay moderate in size; and for one-off emergencies such as snow clearing, a fast near-optimal answer can be worth more than a slow proven one. What has changed is that, for the CPP itself, a fast answer and a proven one are now the same answer.

The natural continuation is the rural postman problem and capacitated routing for fleets, using the same code base, the same benchmark files, and real street networks from OpenStreetMap.

References

Problem and algorithms

- Kwan Mei-Ko (1962). Graphic programming using odd or even points. *Chinese Mathematics* 1, 273–277.
- Edmonds, J. (1965). Paths, trees, and flowers. *Canadian Journal of Mathematics* 17, 449–467.
- Edmonds, J. (1965). Maximum matching and a polyhedron with 0,1-vertices. *Journal of Research of the National Bureau of Standards* 69B, 125–130.
- Edmonds, J. and Johnson, E. L. (1973). Matching, Euler tours and the Chinese postman. *Mathematical Programming* 5, 88–124.
- Hierholzer, C. (1873). Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. *Mathematische Annalen* 6, 30–32.
- Gabow, H. N. (1976). An efficient implementation of Edmonds' algorithm for maximum matching on graphs. *Journal of the ACM* 23, 221–234.
- Galil, Z., Micali, S. and Gabow, H. (1986). An O(EV log V) algorithm for finding a maximal weighted matching in general graphs. *SIAM Journal on Computing* 15, 120–130.
- Kolmogorov, V. (2009). Blossom V: a new implementation of a minimum cost perfect matching algorithm. *Mathematical Programming Computation* 1, 43–67.
- Higgott, O. and Gidney, C. (2023). Sparse Blossom: correcting a million errors per core second with minimum-weight matching. arXiv:2303.15933.

Arc routing

- Corberán, Á. and Laporte, G., eds. (2014). *Arc Routing: Problems, Methods, and Applications*. SIAM, Philadelphia.
- Eiselt, H. A., Gendreau, M. and Laporte, G. (1995). Arc routing problems, part I: the Chinese postman problem. *Operations Research* 43, 231–242.

Benchmark instances

- Golden, B. L., DeArmon, J. S. and Baker, E. K. (1983). Computational experiments with algorithms for a class of routing problems. *Computers & Operations Research* 10, 47–59. (gdb)
- Benavent, E., Campos, V., Corberán, Á. and Mota, E. (1992). The capacitated arc routing problem: lower bounds. *Networks* 22, 669–690. (val)

- Li, L. Y. O. and Eglese, R. W. (1996). An interactive algorithm for vehicle routeing for winter-gritting. *Journal of the Operational Research Society* 47, 217–228. (egl)
- Beullens, P., Muyldermans, L., Cattrysse, D. and Van Oudheusden, D. (2003). A guided local search heuristic for the capacitated arc routing problem. *European Journal of Operational Research* 147, 629–643. (C-F)
- Brandão, J. and Eglese, R. (2008). A deterministic tabu search algorithm for the capacitated arc routing problem. *Computers & Operations Research* 35, 1112–1126. (egl-g)
- Martinelli, R. CARPData.jl: classical CARP instances in a common format. github.com/rafaelmartinelli/CARPData.jl (MIT licence; files used as distributed).

Software

- Hagberg, A., Schult, D. and Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of SciPy 2008*, 11–15.

Original project

- Monserrat Pons, M. (1997). *Resolución del problema del cartero chino (CPP) mediante la aplicación de un proceso de branch and bound*. Final-year project, ETSEIB, Barcelona. Supervisor: R. Companys Pascual.